LAPORAN
WORKSHOP SISTEM INFORMASI
WEB SERVER
(TIF330805)
SEMESTER III



Acara 17

Nama:

Nila Agustin Prima Aurelia

NIM:

E41250666

Golongan:

A

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

Bab 1. Pendahuluan

1.1 Latar Belakang

Sebuah API tidak hanya dinilai dari kemampuannya mengolah data, tetapi juga dari cara API tersebut menyampaikan hasil pemrosesan kepada client. Client seperti aplikasi mobile, aplikasi web lain, atau layanan pihak ketiga harus dapat memahami apakah sebuah request berhasil, gagal karena kesalahan data, atau gagal karena masalah di sisi server. Informasi ini disampaikan melalui kode status HTTP dan struktur JSON pada response.

Pada Acara 17, API CRUD data mahasiswa telah dibangun dengan pola Model-View-Controller (MVC) yang dilengkapi Router, autoloader, dan Front Controller. Pada struktur tersebut, setiap Controller masih membentuk response JSON sendiri melalui method `respond()` yang berisi pemanggilan `http_response_code()`, `header()`, `json_encode()`, dan `exit`. Router dan class Database juga menulis response error dengan cara masing-masing. Akibatnya kode pembentuk response terulang di banyak tempat dan format response belum benar-benar seragam.

Acara 18 menyelesaikan masalah tersebut dengan menambahkan satu class baru pada lapisan Core, yaitu `Response`, sebagai satu-satunya tempat pembentukan response JSON. Controller, Router, dan Database cukup memanggil `Response::json()`, sehingga seluruh response API memiliki format standar (response envelope) dan kode status yang konsisten. Selain itu, validasi data diperjelas dengan kode status 400 untuk data yang tidak lengkap dan 422 untuk data yang lengkap tetapi gagal validasi.

Praktikum ini tetap memakai project si-akademik dan database si_akademik dari acara sebelumnya, tanpa mengubah lapisan Model maupun logika Router.

1.2 Tujuan Praktikum

1. Memahami fungsi `json_encode()` dan `json_decode()` beserta flag `JSON_PRETTY_PRINT`, `JSON_UNESCAPED_SLASHES`, dan `JSON_UNESCAPED_UNICODE` pada PHP Native.
2. Memahami taksonomi kode status HTTP dan menerapkannya pada API menggunakan `http_response_code()`.
3. Membuat class `Response` pada lapisan Core untuk membentuk response JSON yang terstandar (response envelope).
4. Merefaktor `MahasiswaController`, `Router`, dan `Database` agar memakai `Response::json()`.
5. Menambahkan validasi data dengan kode status 400 dan 422.
6. Menguji seluruh endpoint, baik skenario berhasil maupun gagal, menggunakan Postman.
7. Menambahkan field `errors` pada response dan membatasi pretty print hanya pada mode pengembangan melalui konstanta `APP_ENV`.

Bab 2. Dasar Teori

2.1 JSON Encoding dan Decoding pada PHP Native

PHP menyediakan fungsi bawaan `json_encode()` untuk mengubah struktur data PHP (array maupun objek) menjadi teks JSON, serta `json_decode()` untuk proses sebaliknya. Berdasarkan dokumentasi resmi PHP Manual, `json_encode()` dapat menerima flag tambahan, antara lain:

- `JSON_PRETTY_PRINT`, untuk memformat hasil agar mudah dibaca manusia.
- `JSON_UNESCAPED_SLASHES`, agar karakter garis miring tidak di-escape.

- JSON_UNESCAPED_UNICODE, agar karakter non-ASCII (misalnya huruf beraksen atau emoji) ditampilkan apa adanya, bukan dalam bentuk kode unicode.

Beberapa flag dapat digabung menggunakan operator bitwise OR (|).

2.2 Taksonomi Kode Status HTTP

HTTP Status Code merupakan kode numerik yang diberikan server sebagai informasi mengenai hasil pemrosesan suatu request. Kode ini menjadi indikator penting apakah permintaan berhasil diproses atau terjadi kesalahan. HTTP Status Code dikelompokkan menjadi lima kategori.

Kategori	Keterangan
1xx	Informational
2xx	Success
3xx	Redirection
4xx	Client Error
5xx	Server Error

Kode status yang sering digunakan pada REST API ditunjukkan pada tabel berikut.

Kode	Nama	Contoh Penggunaan
200	OK	Permintaan GET/PUT/DELETE berhasil diproses
201	Created	Data baru berhasil dibuat melalui POST
204	No Content	Berhasil tanpa mengembalikan isi
400	Bad Request	Format data pada request tidak valid
401	Unauthorized	Permintaan tidak menyertakan autentikasi yang sah
403	Forbidden	Tidak memiliki hak akses
404	Not Found	Resource atau data yang diminta tidak ditemukan
405	Method Not Allowed	Method tidak diperbolehkan
409	Conflict	Terjadi konflik data
422	Unprocessable Entity	Data valid secara format namun gagal validasi bisnis
500	Internal Server Error	Terjadi kegagalan tak terduga pada sisi server

Pada PHP Native, kode status dikirim menggunakan fungsi http_response_code(), misalnya http_response_code(200); atau http_response_code(404);.

Penerapan HTTP Status Code secara konsisten memberikan beberapa keuntungan, yaitu:

- mempermudah proses debugging;
- meningkatkan interoperabilitas antar aplikasi;
- membantu client mengambil keputusan berdasarkan hasil request;

- mengikuti standar komunikasi HTTP yang telah ditetapkan.

2.3 Standardized Response Envelope

Praktik terbaik dalam desain JSON API merekomendasikan penggunaan format response yang konsisten atau disebut response envelope. Envelope umumnya berisi indikator keberhasilan (`success` atau `status`), data hasil (`data`), pesan (`message`), dan metadata tambahan seperti `timestamp`. Konsistensi format ini memudahkan sisi client dalam memproses response tanpa perlu menangani banyak variasi struktur data untuk kasus yang berbeda-beda.

Response error sebaiknya menyertakan kode error yang unik (bukan hanya pesan teks), rincian tiap field yang gagal validasi, serta metadata seperti request id untuk mempermudah proses debugging pada sistem berskala besar.

2.4 RFC 9457 - Problem Details for HTTP APIs

RFC 9457 yang diterbitkan pada Maret 2023 merupakan standar IETF yang mendefinisikan format JSON baku untuk response error HTTP API. Standar ini memiliki lima bidang utama, yaitu `type`, `title`, `status`, `detail`, dan `instance`, dan disajikan menggunakan tipe konten `application/problem+json`. Meskipun pada praktikum ini digunakan format response envelope yang lebih sederhana, mahasiswa perlu memahami bahwa terdapat standar formal seperti RFC 9457 yang dapat diadopsi pada pengembangan API tingkat lanjut atau API publik berskala industri.

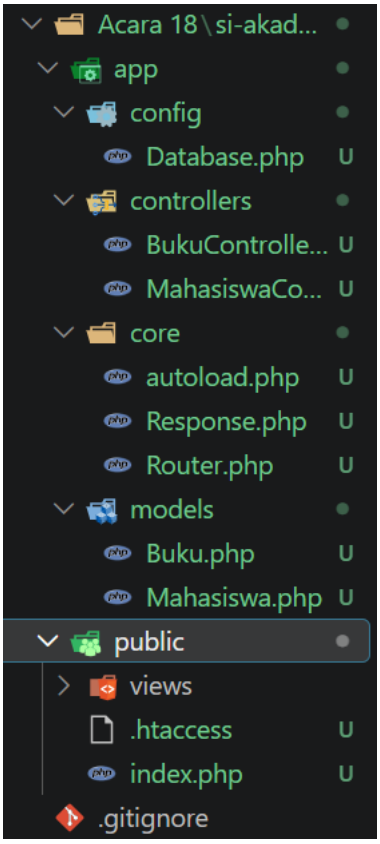
Bab 3. Hasil Praktikum

3.1 Prosedur Kerja

Prosedur kerja acara ini menambahkan satu class baru pada lapisan Core (`app/core/Response.php`) dan merefaktor `MahasiswaController.php` agar menggunakannya, tanpa mengubah lapisan Model maupun logika Router.

a. Persiapan Lingkungan Kerja

Praktikum dilanjutkan pada project si-akademik dari Acara 17 dengan database si_akademik (tabel mahasiswa, prodi, matakuliah, dan buku). Laragon (Apache dan MySQL) dijalankan, kemudian Postman dipakai untuk pengujian.



b. Membuat Class Response pada Lapisan Core

Berkas `Response.php` ditambahkan pada folder `app/core/` sehingga otomatis ditemukan oleh autoloader yang sudah mencakup folder core sejak Acara 17. Class ini memiliki satu method statis `json()` yang menerima kode status, indikator keberhasilan, pesan, data, dan daftar error. Method ini mengirim kode status dengan `http_response_code()`, mengatur header `Content-Type: application/json`, menyusun envelope (`success`, `message`, `data`, `timestamp`), mengubahnya menjadi JSON dengan `json_encode()` beserta flag `JSON_UNESCAPED_SLASHES` dan `JSON_UNESCAPED_UNICODE`, lalu mengakhiri proses dengan `exit`. Nilai `timestamp` dibuat dengan `date('c')`.

Acara 18 > si-akademik > app > core > Response.php > Response

```php
<?php

17 references | 0 implementations
class Response
{
    17 references | 0 overrides
    public static function json(
        int $code,
        bool $success,
        string $message,
        mixed $data = null,
        array $errors = []
    ): void {
        // Memicu autoload Database.php agar konstanta APP_ENV tersedia
        class_exists('Database');

        http_response_code($code);
        header('Content-Type: application/json; charset=utf-8');

        $body = [
            'success' => $success,
            'message' => $message,
            'data'    => $data,
        ];

        if (!empty($errors)) {
            $body['errors'] = $errors;
        }

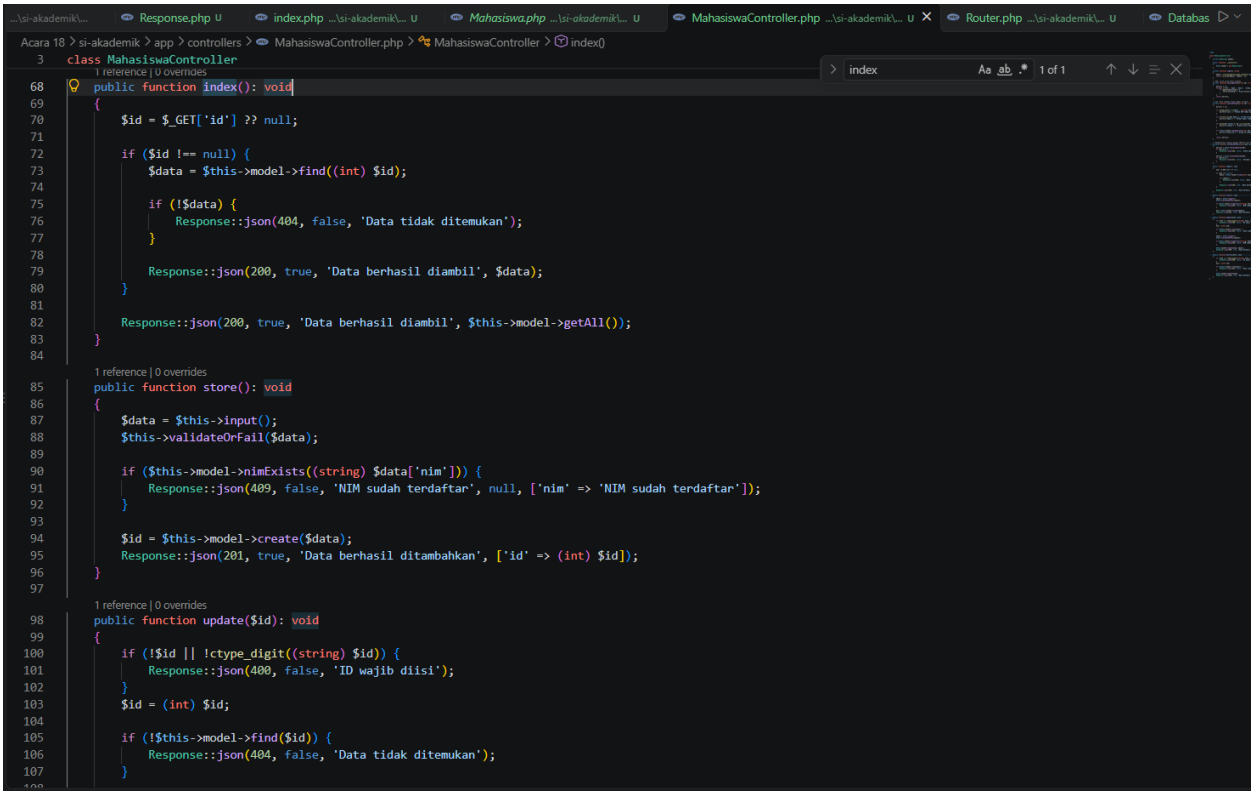
        $body['timestamp'] = date('c');

        $flags = JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE;
        if (APP_ENV === 'development') {
            $flags |= JSON_PRETTY_PRINT;
        }

        echo json_encode($body, $flags);
        exit;
    }
}
```

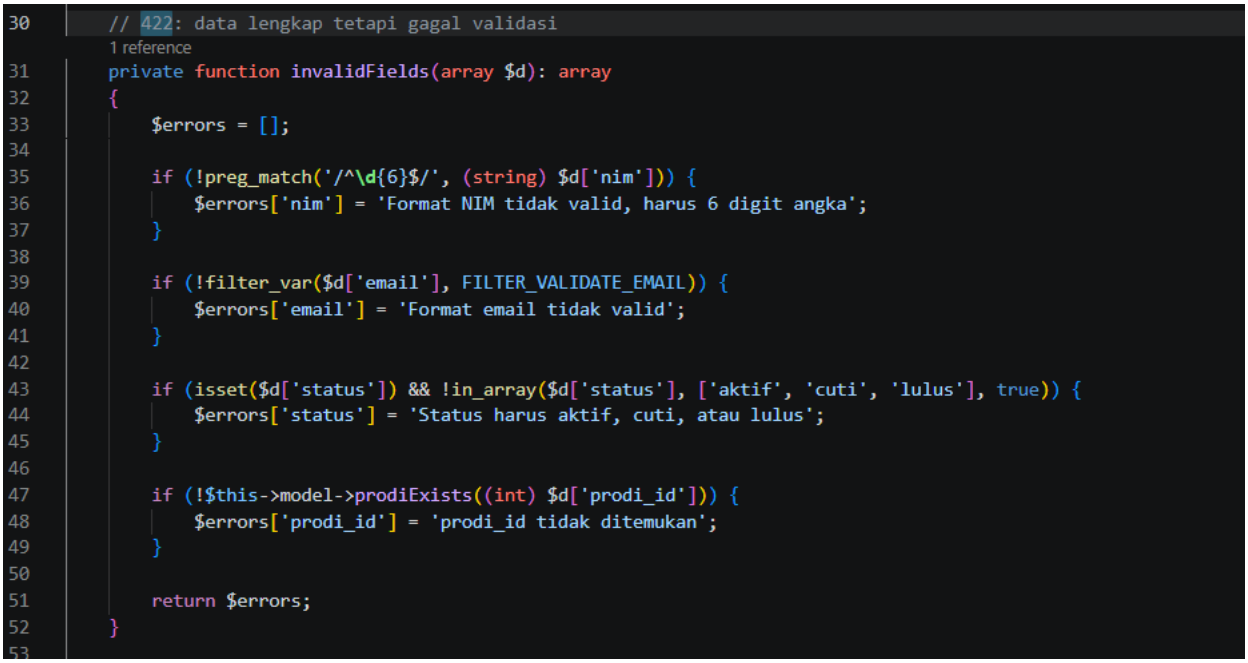
c. Merefaktor MahasiswaController.php

Method privat `respond()` dihapus dan seluruh pemanggilan `http_response_code()` serta `echo json_encode()` pada Controller diganti dengan `Response::json()`. Controller menjadi lebih ringkas karena detail pembentukan JSON kini menjadi tanggung jawab lapisan Core. Perubahan ini berlaku pada method `index()`, `store()`, `update()`, dan `destroy()`.



d. Menambahkan Validasi Data dengan Kode 422

Validasi pada store() dan update() dibagi menjadi dua tahap. Tahap pertama memeriksa kelengkapan field wajib (nim, nama, email, prodi_id, angkatan); jika ada yang kosong, API mengembalikan kode 400. Tahap kedua memeriksa kebenaran isi data; jika gagal, API mengembalikan kode 422. Aturan pada tahap kedua adalah NIM harus terdiri dari 6 digit, format email harus valid, status harus berupa aktif, cuti, atau lulus, dan prodi_id harus terdaftar pada tabel prodi. Pemeriksaan NIM duplikat tetap mengembalikan kode 409.



```php
    // 422: data lengkap tetapi gagal validasi
    1 reference
    private function invalidFields(array $d): array
    {
        $errors = [];

        if (!preg_match('/^\d{6}$/', (string) $d['nim'])) {
            $errors['nim'] = 'Format NIM tidak valid, harus 6 digit angka';
        }

        if (!filter_var($d['email'], FILTER_VALIDATE_EMAIL)) {
            $errors['email'] = 'Format email tidak valid';
        }

        if (isset($d['status']) && !in_array($d['status'], ['aktif', 'cuti', 'lulus'], true)) {
            $errors['status'] = 'Status harus aktif, cuti, atau lulus';
        }

        if (!$this->model->prodiExists((int) $d['prodi_id'])) {
            $errors['prodi_id'] = 'prodi_id tidak ditemukan';
        }

        return $errors;
    }
```

e. Merapikan Response pada Router dan Database

Blok 404 (endpoint tidak ditemukan) dan 405 (method tidak diizinkan) pada app/core/Router.php diubah agar memanggil Response::json(). Blok catch pada app/config/Database.php untuk kegagalan koneksi juga diubah dengan cara yang sama (kode 500). Dengan demikian seluruh response API, baik dari Router, Controller, maupun Database, memiliki format yang seragam.

```php
2 references | 0 implementations
class Router
{
    // nama resource di URL => nama class Controller
    2 references
    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController',
    ];

    2 references | 0 overrides
    public function dispatch(): void
    {
        $uri      = trim(parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH), '/');
        $segments = explode('/', $uri);
        $resource = end($segments) ?: 'mahasiswa';

        $controllerName = $this->map[$resource] ?? null;

        if (!$controllerName) {
            Response::json(404, false, 'Endpoint tidak ditemukan');
        }

        $controller = new $controllerName();

        switch ($_SERVER['REQUEST_METHOD']) {
            case 'GET':
                $controller->index();
                break;
            case 'POST':
                $controller->store();
                break;
            case 'PUT':
                $controller->update($_GET['id'] ?? null);
                break;
            case 'DELETE':
                $controller->destroy($_GET['id'] ?? null);
                break;
            default:
                Response::json(405, false, 'Method tidak diizinkan');
        }
    }
}
```

```php
<?php

// Mode aplikasi: 'development' (JSON pretty print) atau 'production' (JSON ringkas)
if (!defined('APP_ENV')) {
    define('APP_ENV', 'development');
}

11 references | 0 implementations
class Database
{
    6 references
    private static ?PDO $connection = null;

    4 references | 0 overrides
    public static function getConnection(): PDO
    {
        if (self::$connection === null) {
            $dsn = 'mysql:host=localhost;dbname=si_akademik;charset=utf8mb4';

            $options = [
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES   => false,
            ];

            try {
                self::$connection = new PDO($dsn, 'root', '', $options);
            } catch (PDOException $e) {
                Response::json(500, false, 'Koneksi database gagal');
            }
        }

        return self::$connection;
    }
}
```

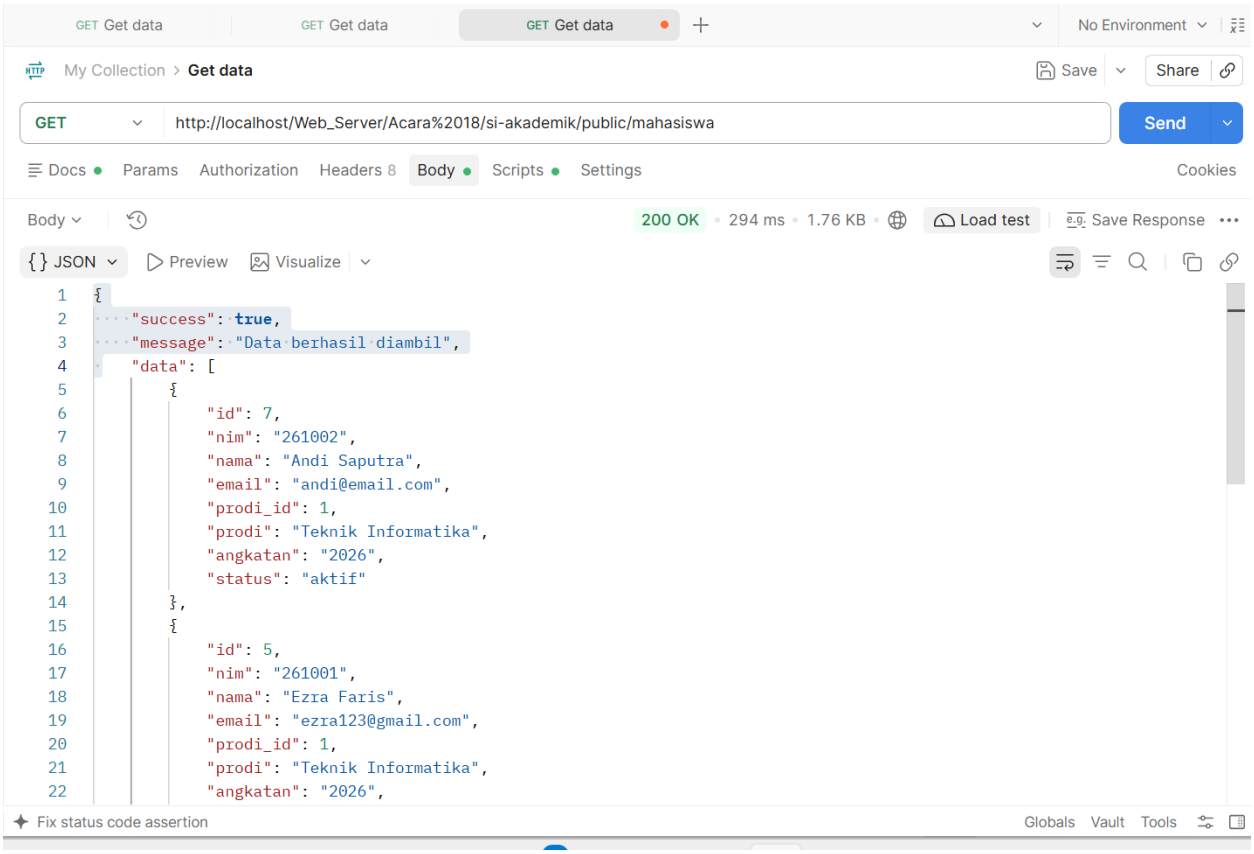
f. Pengujian Ulang Seluruh Endpoint dengan Postman

Pengujian dilakukan pada base URL `http://localhost/Web_Server/Acara%2018/si-akademik/public`. Setiap endpoint diuji ulang dan kode status dicocokkan dengan tabel taksonomi HTTP. Selanjutnya skenario gagal diuji untuk memastikan `Response::json()` mengembalikan kode 400, 404, 409, atau 422 beserta pesan yang informatif.

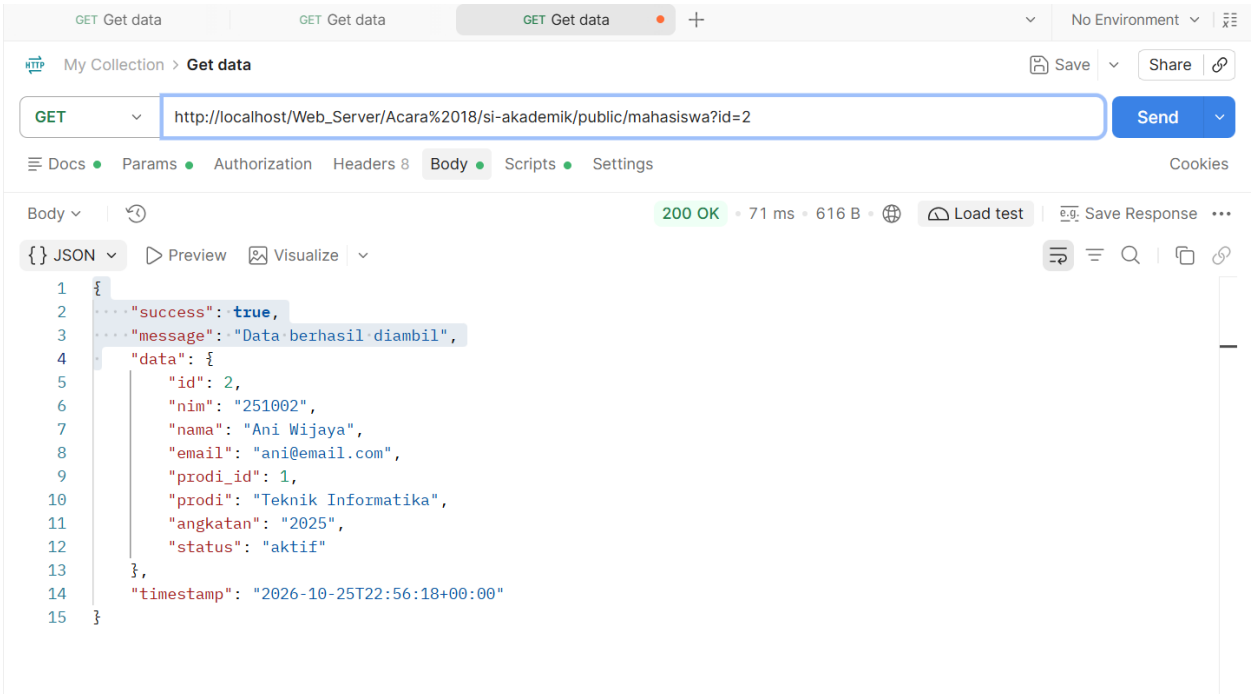
No	Skenario	Request	Status yang Diharapkan
1	Ambil semua data	GET /mahasiswa	200 OK
2	Ambil data berdasarkan id	GET /mahasiswa?id=1	200 OK
3	Id tidak ditemukan	GET /mahasiswa?id=9999	404 Not Found
4	Tambah data valid	POST /mahasiswa	201 Created
5	Body kosong	POST /mahasiswa	400 Bad Request
6	NIM bukan 6 digit atau email salah	POST /mahasiswa	422 Unprocessable Entity

No	Skenario	Request	Status yang Diharapkan
7	NIM sudah terdaftar	POST /mahasiswa	409 Conflict
8	Ubah data valid	PUT /mahasiswa?id=...	200 OK
9	Hapus data	DELETE /mahasiswa?id=...	200 OK
10	Endpoint tidak ada	GET /mahasiswax	404 Not Found
11	Method tidak didukung	PATCH /mahasiswa	405 Method Not Allowed

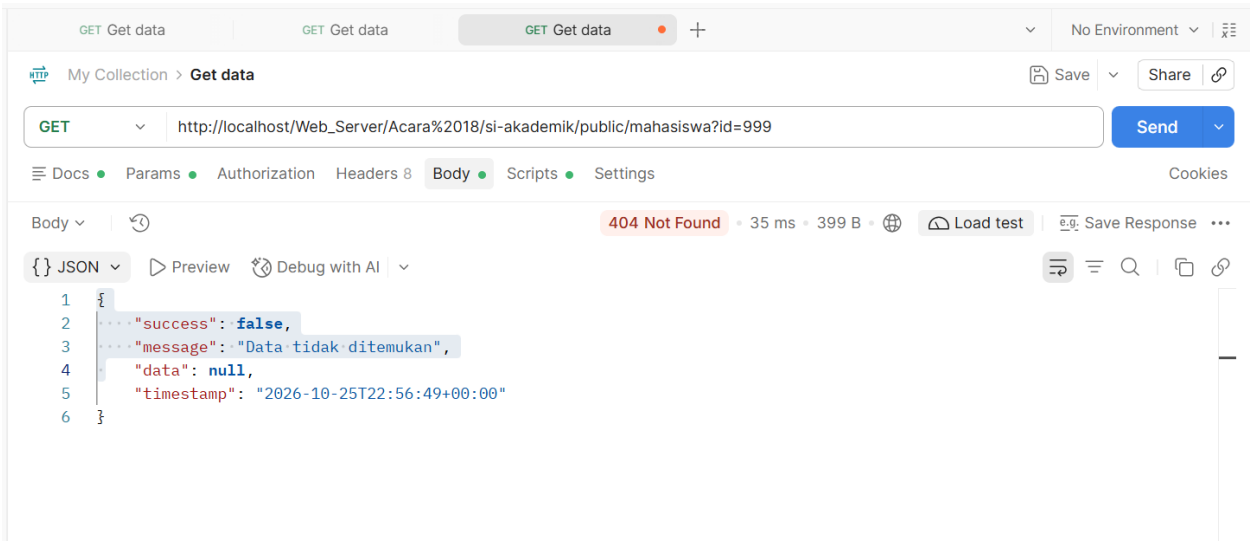
- GET /mahasiswa



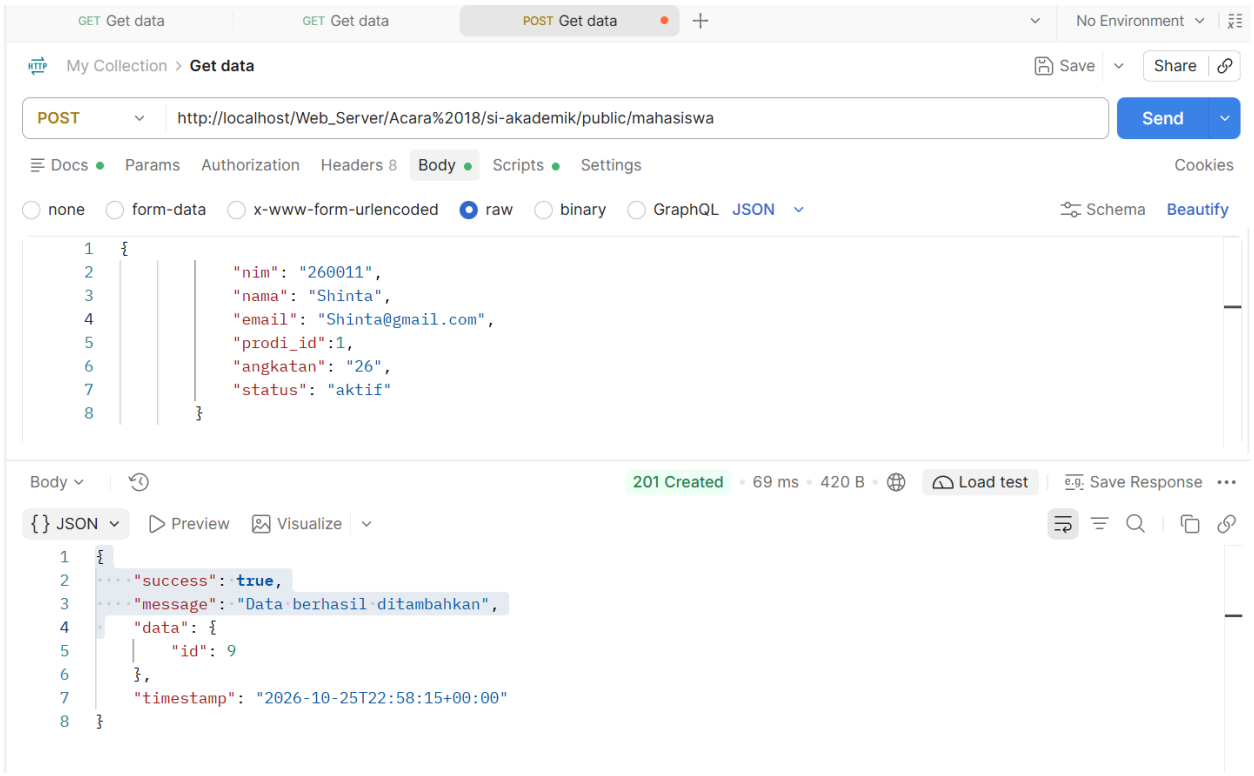
- GET /mahasiswa?id=2



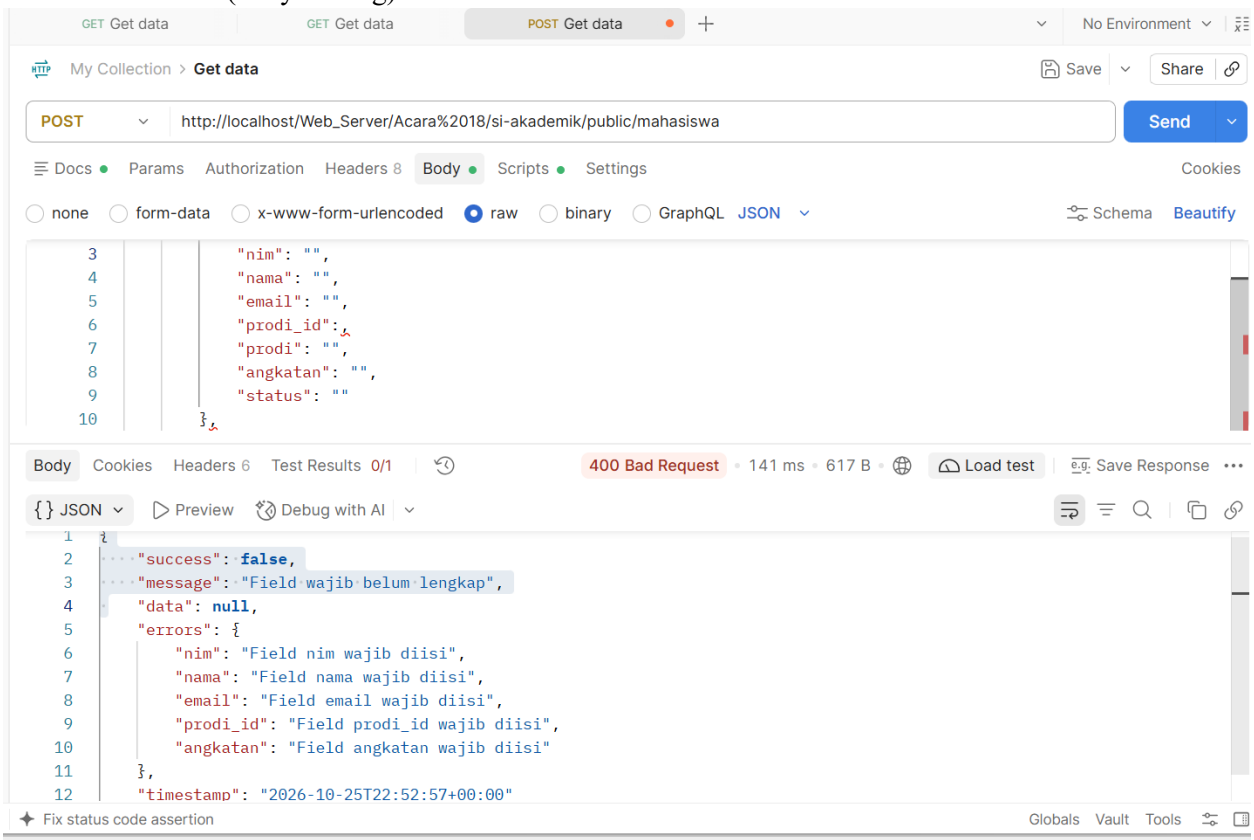
- GET /mahasiswa?id=9999 (id tidak ditemukan)



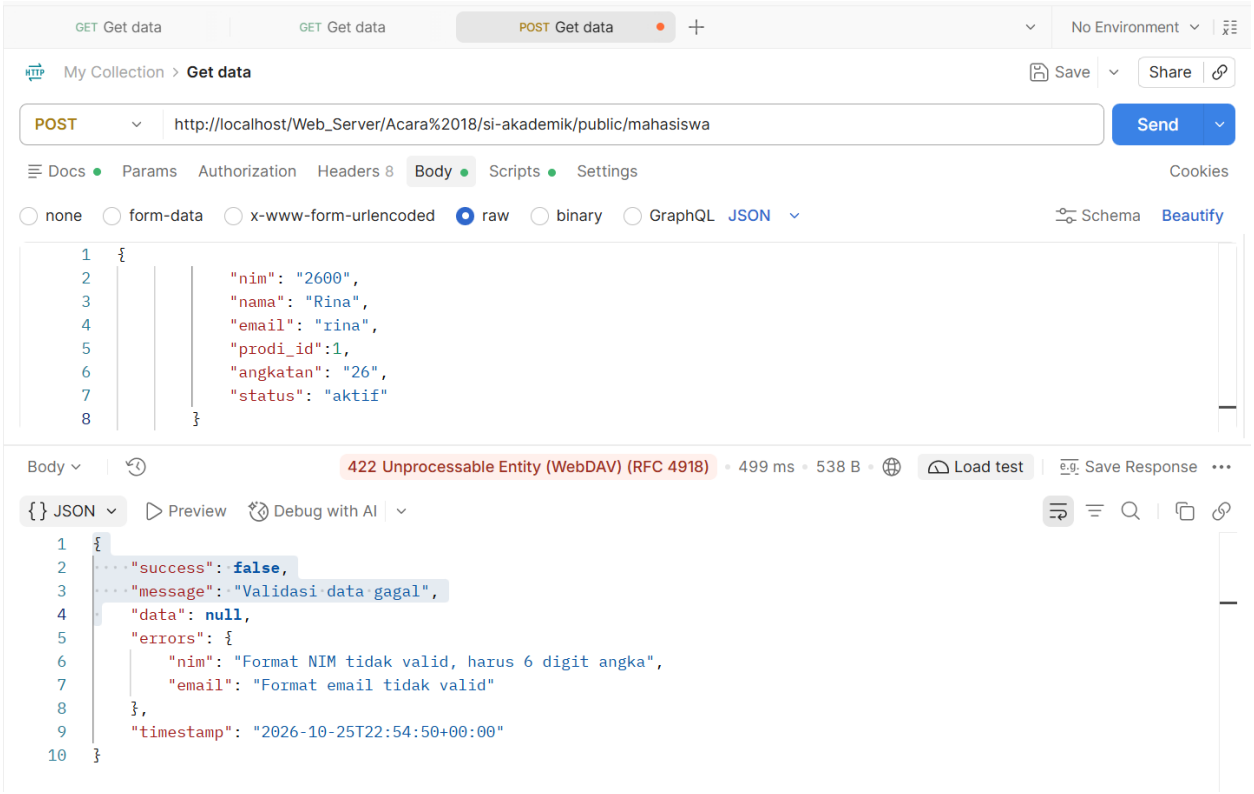
- POST /mahasiswa (data valid)

- 

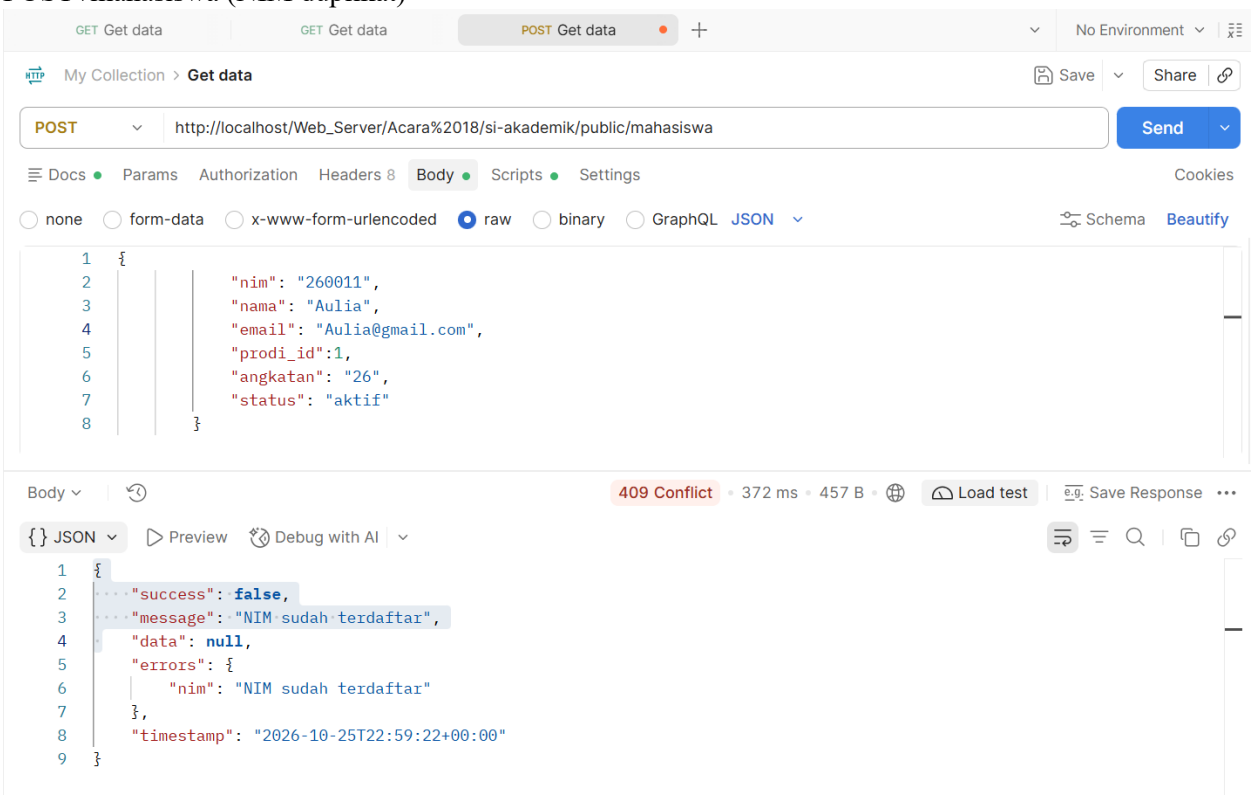
POST /mahasiswa (body kosong)

- 

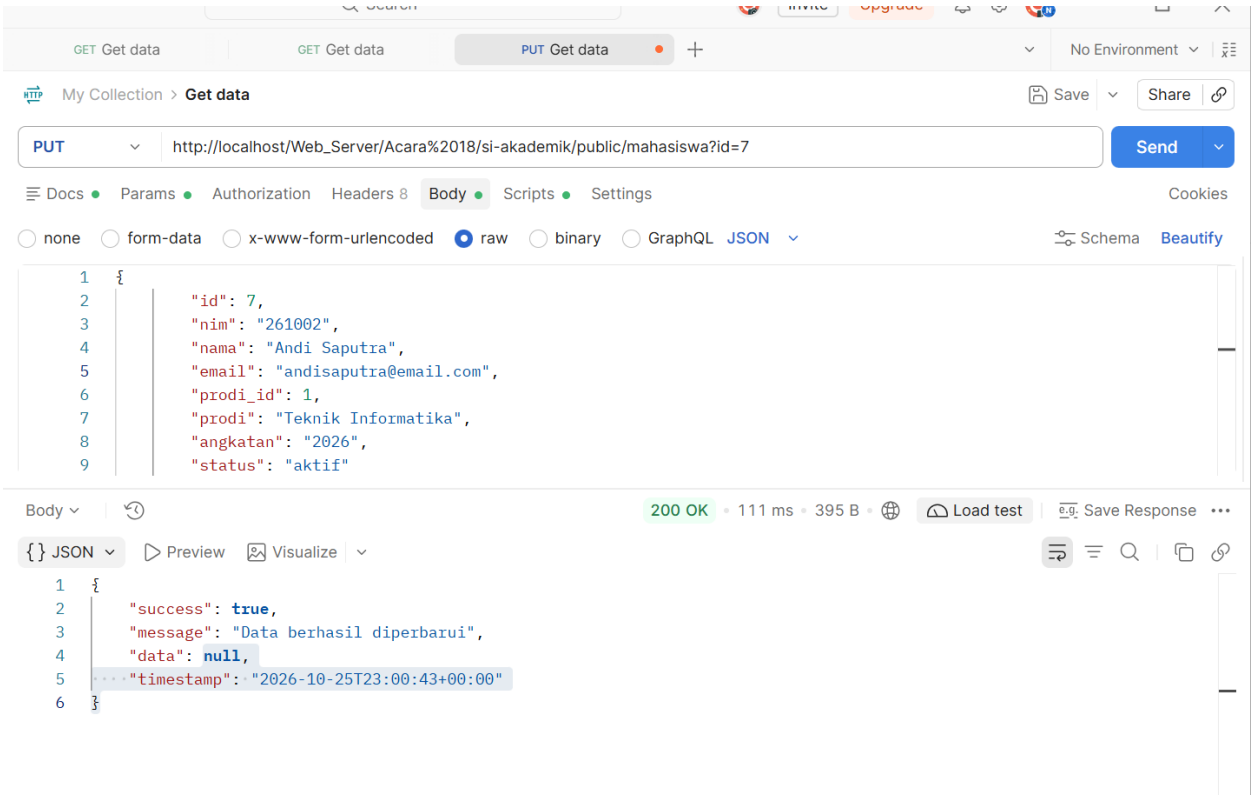
POST /mahasiswa (NIM bukan 6 digit)

- 

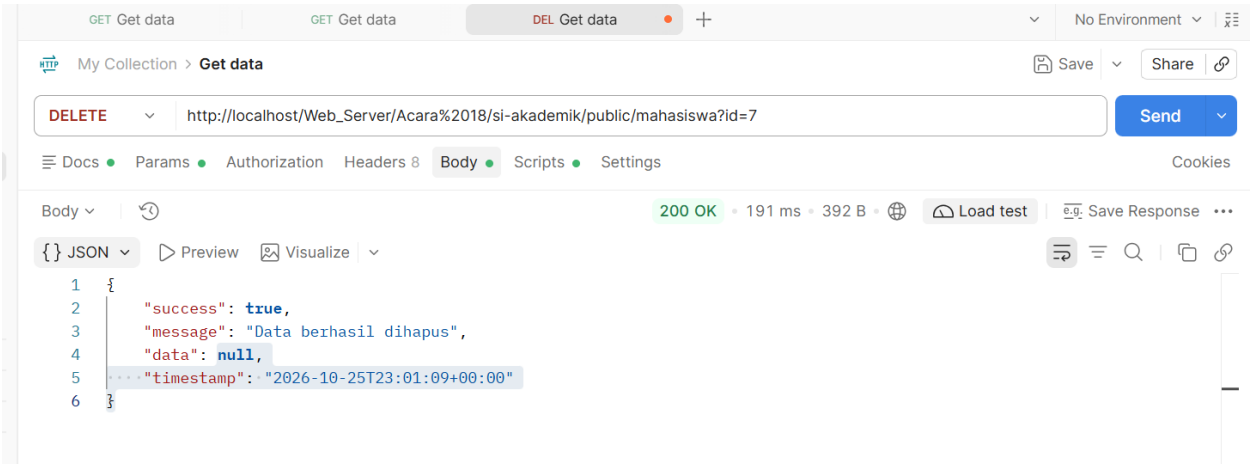
POST /mahasiswa (NIM duplikat)

- 

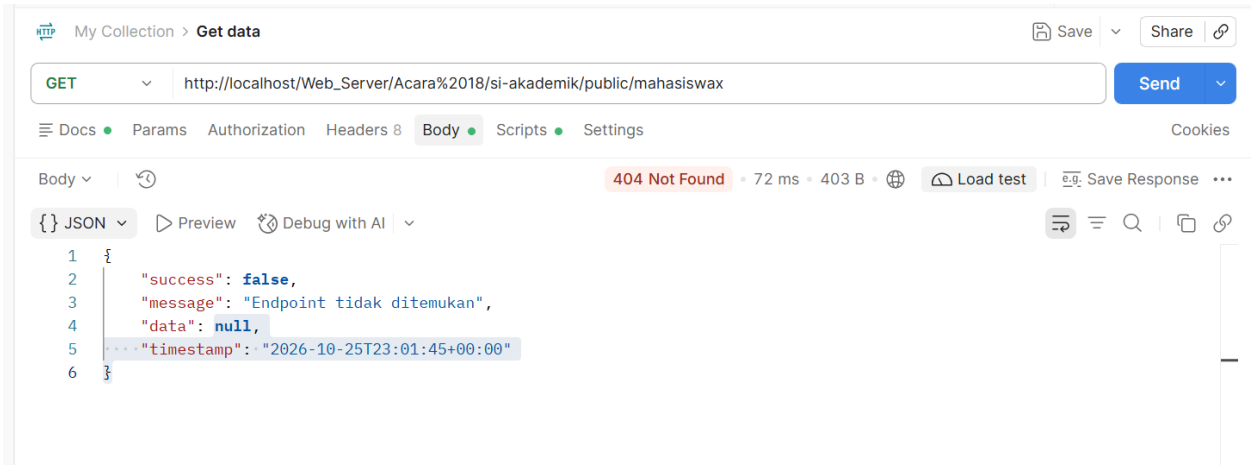
PUT /mahasiswa?id=...



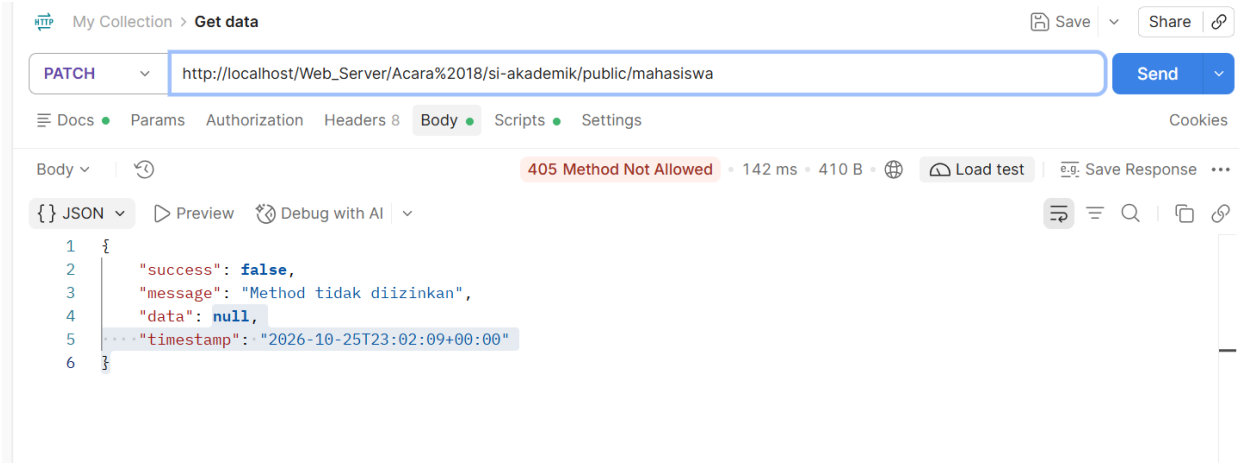
- DELETE /mahasiswa?id=...



- GET /mahasiswax (endpoint tidak ditemukan)



- PATCH /mahasiswa (method tidak diizinkan)



3.2 Tugas Mandiri

a. Menambahkan Field errors pada Response::json()

Parameter `$errors` (array) ditambahkan pada method `Response::json()` untuk menampung daftar pesan kesalahan validasi per field. Field `errors` hanya disertakan pada response apabila array tersebut tidak kosong, sehingga response sukses tetap ringkas. Pada Controller, setiap aturan validasi yang gagal dicatat dengan nama field sebagai kunci dan pesan kesalahan sebagai nilainya. Dengan cara ini client dapat mengetahui seluruh field yang bermasalah dalam satu response, tidak hanya kesalahan pertama.

Acara 18 > si-akademik > app > core > Response.php > Response

```php
<?php

17 references | 0 implementations
class Response
{
    17 references | 0 overrides
    public static function json(
        int $code,
        bool $success,
        string $message,
        mixed $data = null,
        array $errors = []
    ): void {
        // Memicu autoload Database.php agar konstanta APP_ENV tersedia
        class_exists('Database');

        http_response_code($code);
        header('Content-Type: application/json; charset=utf-8');

        $body = [
            'success' => $success,
            'message' => $message,
            'data'    => $data,
        ];

        if (!empty($errors)) {
            $body['errors'] = $errors;
        }

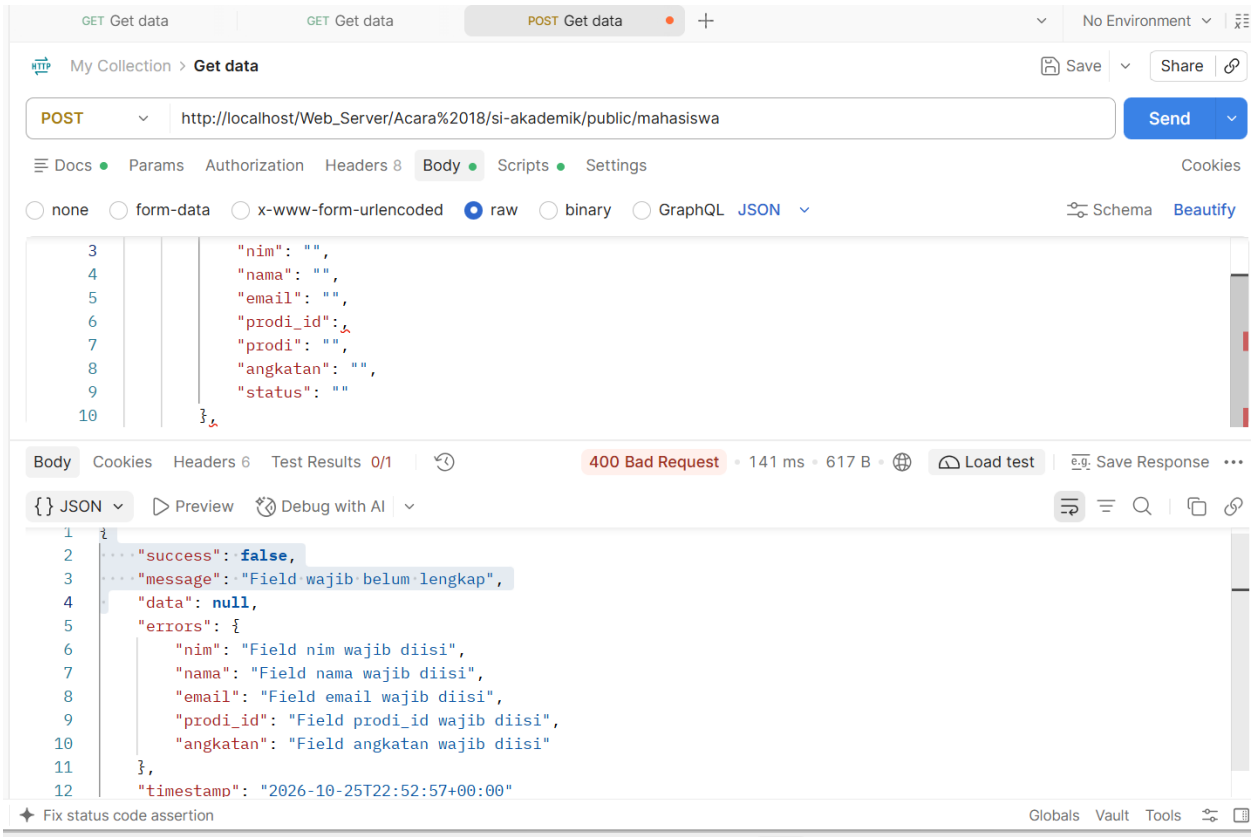
        $body['timestamp'] = date('c');

        $flags = JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE;
        if (APP_ENV === 'development') {
            $flags |= JSON_PRETTY_PRINT;
        }

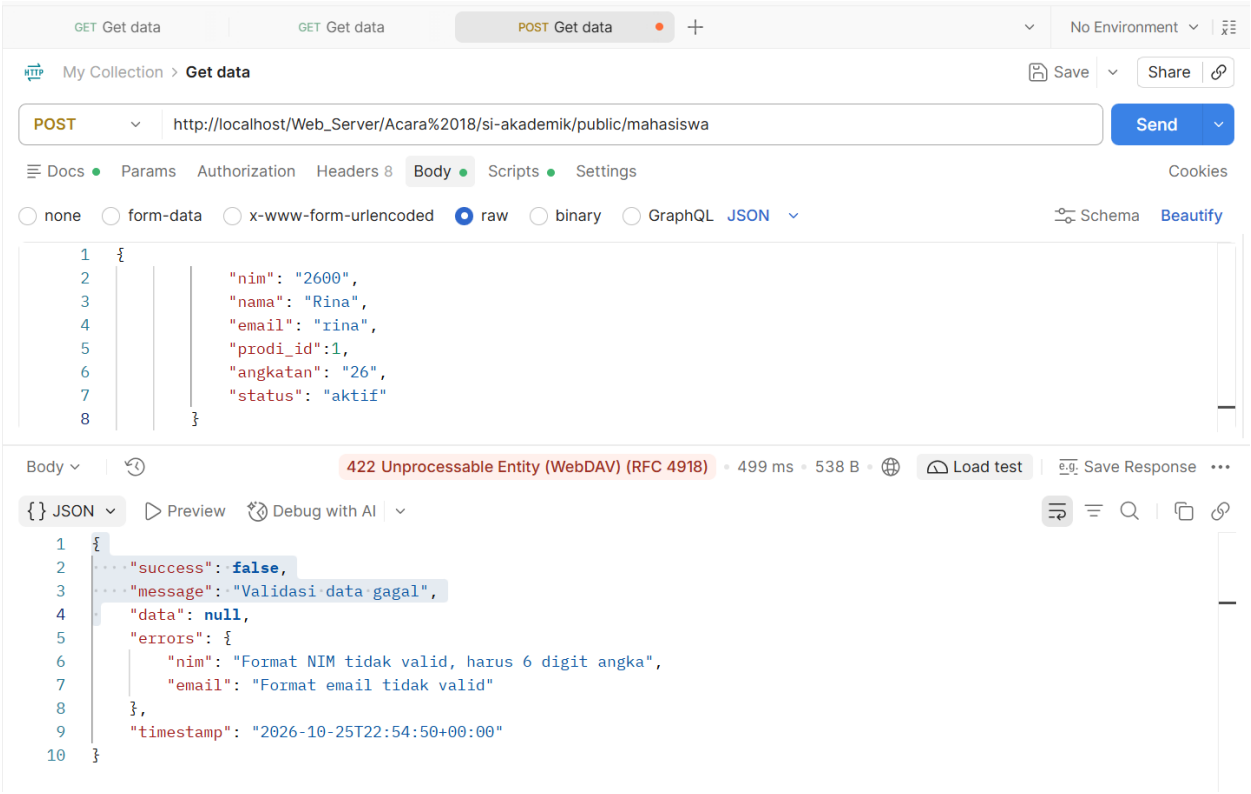
        echo json_encode($body, $flags);
        exit;
    }
}
```

Pengujian Postman:

- POST /mahasiswa dengan body kosong, errors berisi lima field wajib.



- POST /mahasiswa dengan NIM dan email tidak valid, errors berisi pesan untuk field nim dan email.



b. Pretty Print Hanya pada Mode Pengembangan (APP_ENV)

Konstanta APP_ENV didefinisikan pada app/config/Database.php dengan nilai development atau production. Pada Response::json(), flag JSON_PRETTY_PRINT hanya ditambahkan ke json_encode() apabila APP_ENV bernilai development. Karena Database.php baru dimuat autoloader ketika class Database dipakai, sedangkan Router dapat memanggil Response lebih dulu (misalnya untuk response 404), maka pada awal Response::json() ditambahkan pemanggilan class_exists('Database') agar autoloader memuat Database.php dan konstanta APP_ENV pasti tersedia.

```php
<?php

// Mode aplikasi: 'development' (JSON pretty print) atau 'production' (JSON ringkas)
if (!defined('APP_ENV')) {
    define('APP_ENV', 'development');
}

11 references | 0 implementations
class Database
{
    6 references
    private static ?PDO $connection = null;

    4 references | 0 overrides
    public static function getConnection(): PDO
    {
        if (self::$connection === null) {
            $dsn = 'mysql:host=localhost;dbname=si_akademik;charset=utf8mb4';

            $options = [
                PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
                PDO::ATTR_EMULATE_PREPARES   => false,
            ];

            try {
                self::$connection = new PDO($dsn, 'root', '', $options);
            } catch (PDOException $e) {
                Response::json(500, false, 'Koneksi database gagal');
            }
        }

        return self::$connection;
    }
}
```

Acara 18 > si-akademik > app > core > Response.php > Response

```php
<?php

17 references | 0 implementations
class Response
{
    17 references | 0 overrides
    public static function json(
        int $code,
        bool $success,
        string $message,
        mixed $data = null,
        array $errors = []
    ): void {
        // Memicu autoload Database.php agar konstanta APP_ENV tersedia
        class_exists('Database');

        http_response_code($code);
        header('Content-Type: application/json; charset=utf-8');

        $body = [
            'success' => $success,
            'message' => $message,
            'data'    => $data,
        ];

        if (!empty($errors)) {
            $body['errors'] = $errors;
        }

        $body['timestamp'] = date('c');

        $flags = JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE;
        if (APP_ENV === 'development') {
            $flags |= JSON_PRETTY_PRINT;
        }

        echo json_encode($body, $flags);
        exit;
    }
}
```

Pengujian Postman:

- Mode development: response tampil rapi berbaris (pretty print).

[Screenshot Postman: response pada mode development]

- Mode production: response tampil dalam satu baris.

[Screenshot Postman: response pada mode production]

Bab 4. Kesimpulan

Praktikum Acara 18 berhasil menyeragamkan seluruh response API pada project si-akademik dengan menambahkan class `Response` pada lapisan Core. Detail pembentukan JSON, yaitu pengiriman kode status, pengaturan header, penyusunan envelope, dan pemanggilan `json_encode()`, kini terpusat pada satu method `Response::json()`. `MahasiswaController`, `Router`, dan `Database` cukup memanggil method tersebut sehingga kode menjadi lebih ringkas, lebih mudah dirawat, dan menghasilkan format response yang sama untuk setiap kasus. Seluruh perubahan dilakukan tanpa mengubah lapisan Model.

Penerapan kode status HTTP yang konsisten membuat hasil setiap request mudah dipahami oleh client. Kode 200 dan 201 menandai keberhasilan, 400 menandai data yang tidak lengkap, 404 menandai data atau endpoint yang tidak ditemukan, 405 menandai method yang tidak diizinkan, 409 menandai data duplikat, 422 menandai data yang lengkap tetapi gagal validasi, dan 500 menandai kegagalan di sisi server. Pembedaan antara kode 400 dan 422 membuat penyebab kegagalan lebih jelas, dan hasil pengujian Postman menunjukkan seluruh skenario berhasil maupun gagal menghasilkan kode status dan pesan yang sesuai.

Pada Tugas Mandiri, penambahan field `errors` memungkinkan client menerima rincian kesalahan validasi per field dalam satu response. Pembatasan `JSON_PRETTY_PRINT` melalui konstanta `APP_ENV` menunjukkan bahwa format keluaran dapat disesuaikan dengan lingkungan: response yang rapi memudahkan pengembangan dan debugging, sedangkan response yang ringkas lebih efisien untuk produksi. Praktikum ini juga memperkenalkan RFC 9457 sebagai standar formal untuk response error, yang dapat menjadi pengembangan lanjutan dari response envelope sederhana yang dipakai pada praktikum ini.